

Reinforcement Learning and Language Model Alignment

VTR UGE 21

DAY 5 · MODULE 5

Direct Preference Optimization and Evaluation

Preference optimisation without a reward model or sampling, compared with RLHF on matched data, and the evaluation of aligned models with judges, intervals and seeds.

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico),
Vel Tech University (Chennai, India)
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose
utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

Overview

RLHF needs a reward model, sampling during training and careful tuning. Direct preference optimisation reaches the same goal with a simple classification loss on the preference pairs themselves [1]. This day derives the idea, compares DPO with RLHF on exactly the same data, and then turns to a question that is harder than it looks: how to tell whether one aligned model is better than another. It compares judges and their biases [2, 3], puts intervals around every win rate and asks how many comparisons and seeds a claim needs [4].

Estimated study time: 6 to 8 hours.

Learning outcomes

By the end of the day, students are expected to derive the DPO loss from the KL-regularised optimum and the Bradley-Terry model, to implement it and explain the role of its gradient weight, to compare DPO with RLHF on gold reward, KL and the quality of the outputs and explain the drift of DPO for small beta, to evaluate aligned models with several judges and recognise a judge that measures a surface property, to treat ties and report bootstrap intervals, and to plan the number of comparisons and seeds needed for a decision. In Python, students are expected to distinguish an alias from a copy of an object, to compute a bootstrap interval and to write a function that returns a dictionary of results.

Python in this day

The lecture is followed by Python step 5: Objects, copies and intervals. It covers how two names can refer to one object and why the reference policy must be a copy, a bootstrap interval for a win rate and a function that returns a dictionary of results [5, 6]. The Colab notebook opens with the same step and continues with DPO and the evaluation.

From the optimum to DPO

The objective of RLHF, reward minus a KL penalty, has a known optimum: The best policy is the fine-tuned model reweighted by the exponential of the reward. Rafailov and colleagues turned this around [1]. If the optimal policy determines the reward, the reward can be written in terms of the policy, and the Bradley-Terry likelihood of the preferences becomes a loss on the policy itself. Direct preference optimisation trains the model on the preference pairs with this loss, without a separate reward model and without sampling new answers during training. Its parameter beta plays the role of the KL weight.

Animation on the lecture page: The DPO loss and its gradient weight against the implicit reward margin of a pair. Change beta and move the margin.

In the Colab notebook, section 1.

Rebuild the world of Day 4: the fine-tuned model, the preference pairs, the reward model and RLHF.

In the Colab notebook, section 2.

Implement the DPO loss and train the model directly on the preference pairs.

Check your understanding (1). What does DPO need that RLHF also needs?

- A. A separate reward model
- B. Sampling during training
- C. Preference pairs
- D. A value function

DPO against RLHF on matched data

A fair comparison gives both methods the same fine-tuned model and the same pairs. In the notebook, DPO reaches a slightly higher gold reward than RLHF but moves further from the fine-tuned model, and a clearly smaller share of its sentences still follows the grammar. With a smaller beta, DPO drifts so far that most sentences break the grammar. Neither method is better in every respect; the comparison depends on what is measured.

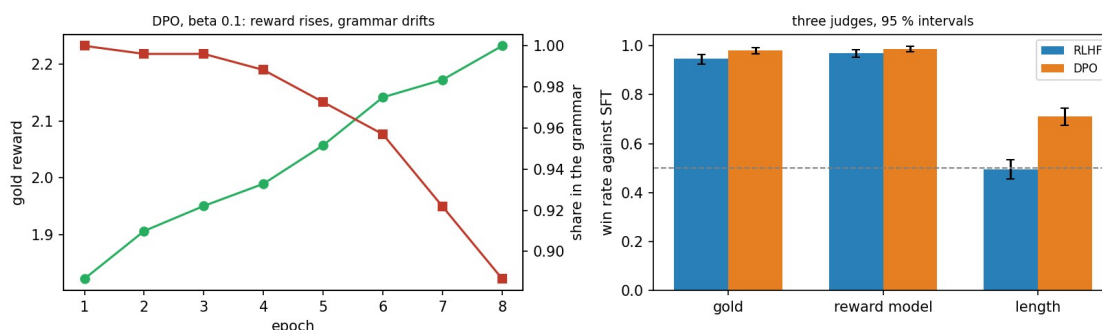


Figure. Left: DPO over eight epochs, with the gold reward rising and the share of grammatical outputs falling. Right: Win rates against SFT under three judges, with 95 percent intervals.

In the Colab notebook, section 3.

Compare DPO and RLHF on matched data: gold reward, distance from the fine-tuned model and grammar.

Check your understanding (2). DPO reaches a higher gold reward than RLHF but breaks the grammar more often. What follows?

- A. DPO is better
- B. RLHF is better
- C. The answer depends on which property matters for the application
- D. Both failed

Judges and their biases

Aligned models are compared by asking a judge which of two answers is better, and reporting a win rate. The judge may be a person, a reward model or another language model [2]. Judges have biases; a well-known one is a preference for longer answers [3]. In the notebook, a judge that simply prefers longer sentences cannot separate RLHF from the fine-tuned model, yet declares DPO clearly better, partly

because DPO writes longer sentences; a judge that prefers short sentences ranks DPO below the fine-tuned model. How ties are counted can also change a conclusion.

In the Colab notebook, section 4.

Compare RLHF and DPO with three judges: the gold reward, the reward model and a judge that counts words.

Check your understanding (3). A judge prefers longer answers. What can it reward?

- A. Better answers only
- B. Answers that are merely longer
- C. Shorter answers
- D. Nothing

Intervals, sample size and seeds

A win rate measured on a few hundred comparisons is an estimate with an interval around it. When the interval contains one half, the comparison is undecided. The notebook trains the same recipe twice with different seeds: With 400 comparisons the two runs cannot be told apart; with 1600 they can, with a win rate barely above one half. A claim that one model is better needs its interval, its number of comparisons and more than one seed [4]. Part B of the lab reads intervals, and Part C computes one by hand.

Animation on the lecture page: The spread of win-rate estimates over 400 simulated experiments. Change the number of comparisons and the true win rate and read off how often an experiment reaches a decision.

In the Colab notebook, section 5.

Put bootstrap intervals around the win rates and find how many comparisons decide between two seeds.

Check your understanding (4). A win rate is 0.52 with a 95 percent interval from 0.47 to 0.57. What can be concluded?

- A. A is better
- B. B is better
- C. The comparison is undecided
- D. The judge is biased

Your judge

The notebook's switch lets you define the judge: one that counts calm words, one that prefers short sentences, one that punishes repetition or one that only checks the opening word. The same two models win clearly under the first judge, lose under the second and third, and tie almost always under the fourth, because it cannot tell the answers apart. A win rate means nothing without its judge.

In the Colab notebook, section 6.

Set the switch to a judge of your choice and compare RLHF and DPO against the fine-tuned model.

Open problems and the final project

Alignment by preferences leaves open questions: whose preferences are collected, how judges can be checked, how a model behaves outside the data it was tuned on, and how to detect reward hacking that a learned reward cannot see [7]. The final project asks for a small alignment study of your own, with every element of the week: a world with a known gold reward, a preference method, an honest evaluation with intervals and seeds, and a discussion of what the gold reward does not capture.

In the Colab notebook, section 7.

Read the brief of the final project and the rubric.

Python step 5: Objects, copies and intervals

Two practical details decide whether an alignment experiment is valid: The reference model must stay frozen while the policy changes, and every win rate needs an interval. This step shows both in plain Python and NumPy [5, 6].

An alias is not a copy

```
import copy
class Policy:
    def __init__(self, weights):
        self.weights = weights

pol = Policy([1.0, 2.0])
alias = pol                # a second name for the same object
frozen = copy.deepcopy(pol) # an independent object with the same content
pol.weights[0] = 99.0
print("alias :", alias.weights)
print("frozen:", frozen.weights)
```

Output

```
alias : [99.0, 2.0]
frozen: [1.0, 2.0]
```

Assignment in Python never copies an object; it gives the object another name. A reference policy created by `ref = pol` would change with every update of the policy, and its KL to the policy would always be zero. `copy.deepcopy`, or the `copy` method of the language model in the notebook, creates an independent object.

A bootstrap interval

```
import numpy as np
wins = np.array([1, 1, 0, 1, 1, 0, 1, 1, 1, 0] * 10, float) # 100 comparisons, 70
wins
rng = np.random.default_rng(0)
boot = rng.choice(wins, (2000, len(wins))).mean(axis=1)      # 2000 resampled win
rates
print("win rate:", wins.mean(), " 95 % interval:", np.percentile(boot, [2.5,
97.5])).round(3))
```

Output

```
win rate: 0.7    95 % interval: [0.61 0.79]
```

`rng.choice(wins, (2000, 100))` draws 2000 rows of 100 comparisons with replacement, and the mean of each row is one resampled win rate. The 2.5th and 97.5th percentiles of these values bound the 95 percent interval.

A function that returns a dictionary

```
def summarise(outcomes, n_boot=2000, seed=0):
    """Win rate of a series of comparisons, with its bootstrap interval."""
    outcomes = np.asarray(outcomes, float)
    boot = np.random.default_rng(seed).choice(outcomes, (n_boot, len(outcomes))).mean(1)
    low, high = np.percentile(boot, [2.5, 97.5])
    return {"win_rate": float(outcomes.mean()), "low": float(low), "high": float(high),
            "n": len(outcomes)}

for n in (20, 100):
    s = summarise(wins[:n])
    print(f"n = {s['n']:3d}: win rate {s['win_rate']:.2f}, interval [{s['low']:.2f}, {s['high']:.2f}]" )
```

Output

```
n = 20: win rate 0.70, interval [0.50, 0.90]
n = 100: win rate 0.70, interval [0.61, 0.79]
```

Returning a dictionary keeps every number of a result together with its name, which makes it easy to collect many evaluations into a data frame. With five times as many comparisons, the interval becomes much narrower.

Quick check. What would go wrong if the reference policy of RLHF were created with `ref = pol`?

Answers to the knowledge checks

1. **C.** DPO learns from the pairs directly; it needs neither a reward model nor sampling.
2. **C.** A single number hides the trade-off.
3. **B.** A length bias makes length look like quality.
4. **C.** The interval contains one half.

Quick check. The reference would be the same object as the policy and change with it, so the KL penalty would always be zero and the leash of Day 4 would disappear.

References

- [1] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (pp. 53728-53741).
- [2] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track.

- [3] Singhal, P., Goyal, T., Xu, J., & Durrett, G. (2024). A long way to go: Investigating length correlations in RLHF. In First Conference on Language Modeling (COLM 2024). arXiv:2310.03716.
- [4] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. In Proceedings of the AAAI Conference on Artificial Intelligence, 32(1), 3207-3214.
- [5] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.
- [6] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.
<https://doi.org/10.1038/s41586-020-2649-2>
- [7] Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., et al. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. Transactions on Machine Learning Research.
- [8] Azar, M. G., Guo, Z. D., Piot, B., Munos, R., Rowland, M., Valko, M., & Calandriello, D. (2024). A general theoretical paradigm to understand learning from human preferences. In Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS 2024), PMLR 238. arXiv:2310.12036. <https://arxiv.org/abs/2310.12036>
- [9] Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., & Kiela, D. (2024). KTO: Model alignment as prospect theoretic optimization. In Proceedings of the 41st International Conference on Machine Learning (ICML 2024). arXiv:2402.01306.

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose

utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com